CSE 350/550: Network Security, Mid-semester exam: Feb 27, 2025, 90 minutes, 90 marks

Instructions:

- The exam is in **two** parts: Part 1 has 5 questions that require short answers. Each question is for 6 marks → 30 marks. Part 2 has 5 questions that require longer answers. Each question is of 12 marks → 60 marks.
- This is a "closed-book" exam. All necessary information, including tables, etc. will be provided here.
- Write your answers in the space provided. If necessary, use a separate sheet for rough work.
- Do not use unfair means. Action will be taken against you if you use unfair means. You may not use books, notes, computer, smart phone, etc. or share info with other students. You may only use a calculator.

Part 1: 5 questions that require short answers (6 marks each)

1. A. What is that property of 'exponentiation modulo N' that allows one to efficiently calculate X^K mod N, where K is possibly a large number?

Marks on Q1: 6

$X^K \bmod N = X^{(K1+K2)} \bmod N = (X^{K1} \bmod N * X^{K2} \bmod N) \bmod N$

Above $K = K1 + K2$

3

b. Illustrate your answer by showing how 88^7 mod 187 is calculated. Note $88^7 \bmod 187 = 88^{4+2+1} \bmod 187$

3 →

$88^2 \bmod 187 = 88 * 88 \bmod 187 = 77$

$88^4 \bmod 187 = 77 * 77 \bmod 187 = 132$

$88^6 \bmod 187 = 132 * 77 \bmod 187 = 66$

$88^7 \bmod 187 = 66 * 88 \bmod 187 = 11$

2. Consider GF(2^4). And assume that the irreducible polynomial m(x, 4) = $x^4 + x + 1$. Then:

Marks on Q2: 6

a. What is the underlying set of symbols in GF(2^4) in hexadecimal terms?

Set of $GF(2^4) = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$

b. What is 4 + C?

$(4+C)_H = 0100 + 1100 = 1000 = (8)_H$

2

4 →

c. What is 4 * C?

$4 * C = 0100 * 1100$

2 polynomial terms $x^2 * (x^3 + x^2) \bmod M(x)$

$= x^5 + x^4 \bmod M(x)$

$= x^2 + 1$ OR $4 * C = (5)_H$

$x^4+x+1 \overline{)\, x^5+x^4\,(x+1}$

x^5+x^2+x

x^4+x^2+x

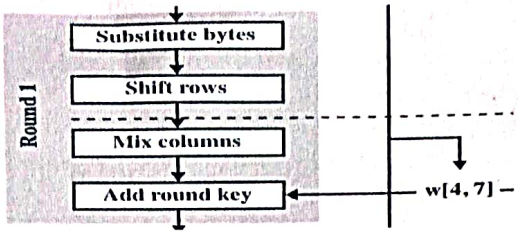
x^4+x+1

x^2+1 OR 0101 or $(5)_H$

3. Consider AES. A portion of the input to 'Substitute Bytes' of the kth round is given below. What is the last row of the output from 'Substitute Box' and similarly from 'Shift Rows'? The S-box table is given below:

Marks on Q3: 6

-	-	-	-	
-	-	-	-	
-	-	-	-	
AC	40	52	B2	



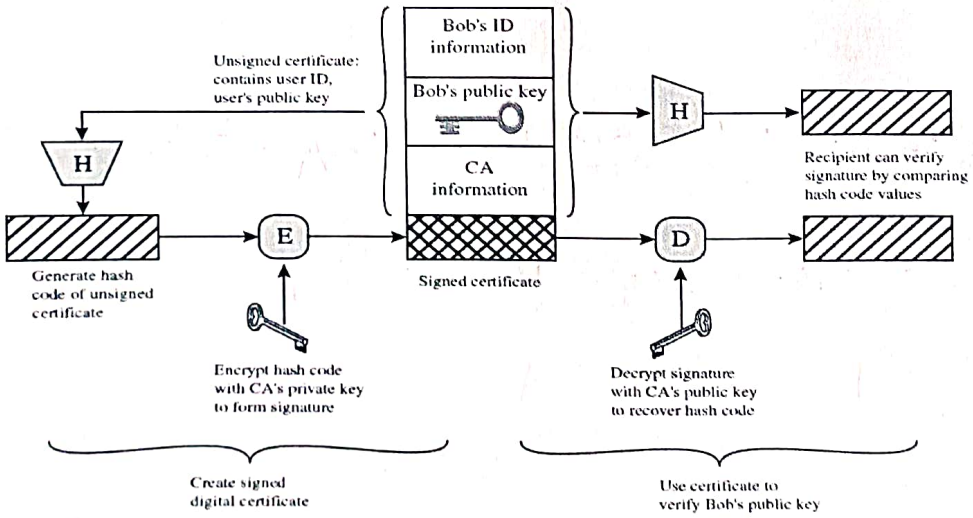
x \ y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

a. Output of 'Substitute Box': 91 09 00 37 ← 3

b. b. Output of 'Shift Rows': 37 91 09 00 ← 3

4. Given below is the processing at end of the 'certification authority' that results in generation of 'public-key certificate', and its verification at the receiver's end. Note, plaintext being 'hashed' is of arbitrary length.

Marks on Q4: 6



In your assessment what properties must the 'hash' function H satisfy to qualify to be a proper hash function for use in generating public-key certificates. To be sure **(CONTINUED ON NEXT PAGE)**:

Continued from question 4 on previous page

3. Hash value should be large no. of bits (such as 32 ~ 64 bits) to minimize collisions

a. List 2 properties.

2 →

1. Fixed no of bits in hash value (such as 32 bits)
2. Computationally very difficult to change the user info without change in hash value

b. Illustrate your answer by showing what constitutes a POOR hash function.

2 →

Break up the INFO into 32 bit blocks and add them up (binary addition) to obtain a 32-bit hash value

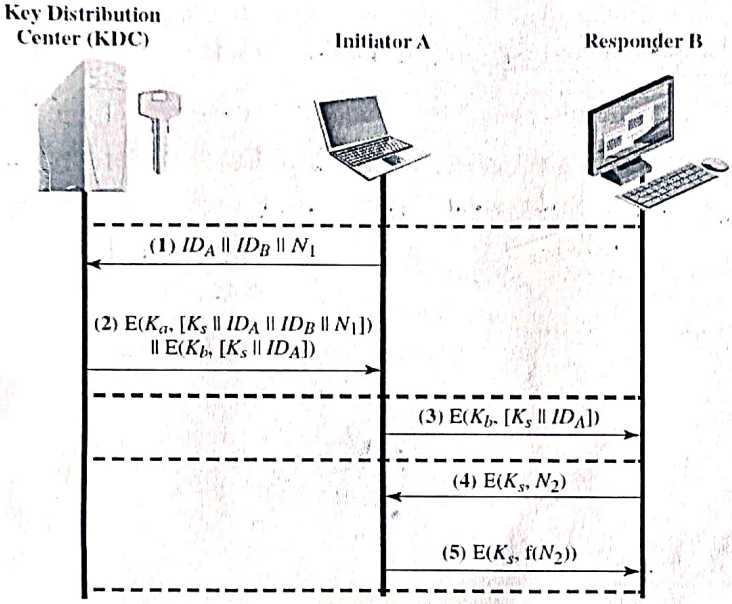
c. Why is error detecting code CRC 32 not a good hash function in generating public-key certificates?

2 →

It is not very difficult to change INFO so that the CRC value is unchanged. It is good for random changes in INFO.

5. As shown below, devices A and B can share a symmetric encryption key, K_s, by obtaining the key from a "key distribution center", KDC, using an existing secure channel between A and KDC, or between B and KDC.

Marks on Q5: 6



A. What purpose do messages numbered (4) and (5) serve?

3 →

Confirm that both 'A' and 'B' have the newly shared key K_s. They confirm this by A & B being able to decode messages (3) & (4).

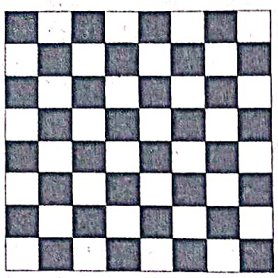
B. In message (5) could $f(N_2)$ be replaced simply by N2? Explain briefly.

3 →

NO. Because ~~the only~~ any other device can 'copy' ~~snoop~~ the message (3) and send it back as (4) without possessing K_s, if $f(N_2)$ is replaced by N_2.

Marks on Q6: 12

6. I have a Black-and-White photo (with no grayscale) of an 8x8 chess board. The photo is 1024x1024 pixels where each pixel is encoded using 8 bits (see sample photo below, but ignore the lines on the edge of the board).



I wish to encrypt this image and communicate the resulting ciphertext confidentially so that an intruder is NOT able to determine that this is a ciphertext corresponding to a chess board.

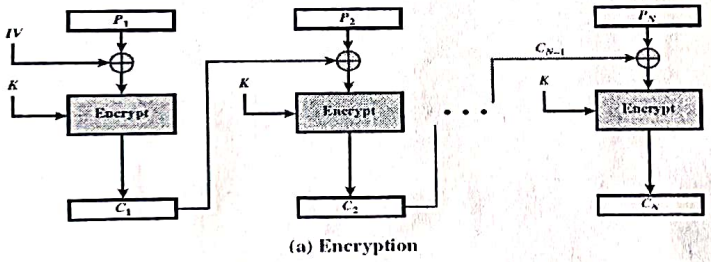
a. How can I do this using a block cipher with or without block chaining or using a stream cipher?

6 If we use DES with block-chaining on 64-bit blocks, the ciphertext corresponding to 64 bit block in row X and column Y will be very very different from the ciphertext corresponding to 64 bit block in row X and column Y+2 (as also other columns, and other rows)

4 → b. Give a block diagram that shows processing at sender's end.

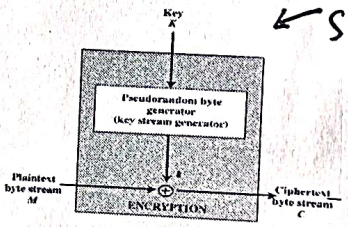
This will be similar if we use a stream cipher

Block Chaining ↓



(a) Encryption

OR



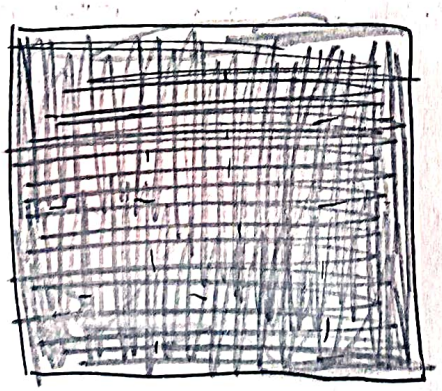
← stream cipher

c. What is the encryption algorithm you propose to use?

DES or AES or RC4 in case of stream cipher.

d. What does the ciphertext look like when your method is used?

2 ↑



Marks on Q7: 12

7. Consider using a 'stream cipher', that encrypts a stream of 16 bit 'words" using a 16-bit key-stream obtained from a pseudo-random generator based on 'linear congruential generator' such as $X_{n+1} = (a X_n + c) \bmod m$. Here constants m, a, c are publicly known, but X_0, the seed, is secret. This works fine as long as an adversary is unable to guess X_k for any k. That is, if an adversary is able to launch an attack and compute X_k for any k then the rest of the ciphertext can be de-ciphered with ease.

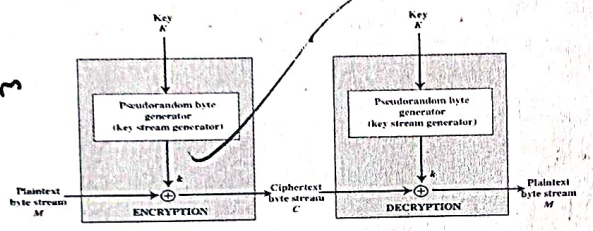
Question: how can this be prevented? HINT: use a key-stream consisting of a sequence of 16-bit words derived from the 16 lowest significant bits (LSB) of a stream of 32-bit random numbers, generated for instance using a 'linear congruential generator', such as $X_{n+1} = a X_n \bmod m$, $m = 2^{31}-1$, $a = 16807$, $X_0 = 771$. The first 4 random numbers, and their 32-bit reps are given below:

771: 00000000 00000000 00000011 00000011
12958197: 00000000 11000101 11011011 01110101
892568632: 00110101 00011110 11011011 01101000
1227723729: 01001000 10010001 11001100 10010001,

Explain why this strategy might work. To answer, you may for example, assume X_2 was discovered. And explain why that is not enough.

16-bit key stream

16-bit key stream



Let X_k, $k=0,1,\ldots$ be the pseudo random no. generated, and Y_k, $k=0,1,\ldots$ be the 16 bit LSB obtained from X_k and used as the key stream.

Now suppose an intruder is able to estimate/compute Y_k for some k, he/she will not be able to estimate/compute Y_{k+1} the next 16-bit word in the key stream. This is so, since he/she will need to estimate/compute the 32-word X_k and the estimate/compute the 16-word Y_k used in key stream. Clearly he/she will not be able to do so.

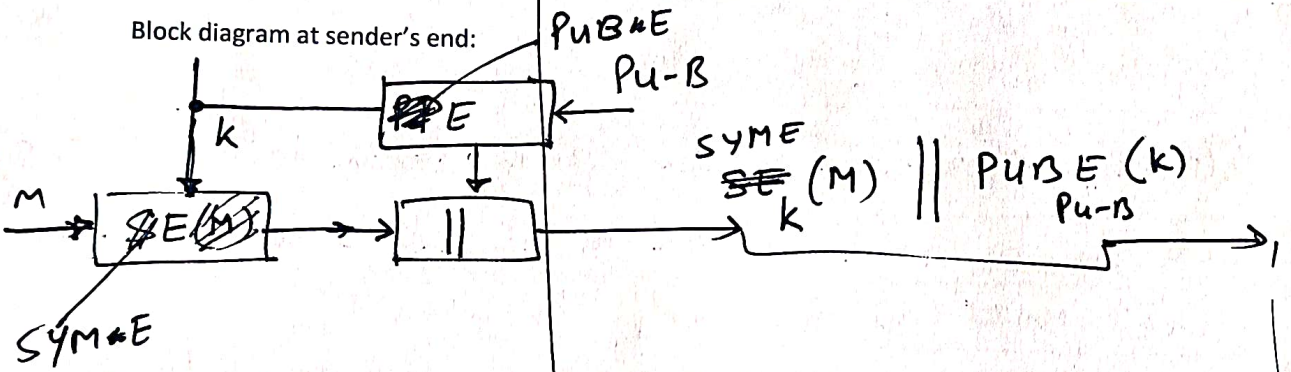
Thus this scheme looks perfectly good.

8. Each PhD student has a RSA related <public-key, private-key> pairs, or <PU-A, PR-A>, <PU-B, PR-B>, etc. They have securely shared their public keys with me. I can now encrypt my messages and confidentially share with student A using his public key, PU-A. However, I find that encrypting a long message (such as 20 MB PPT file) using the public-key crypto-system takes a long time. The advantage of using public-key cryptography is anyone can send a message to student A in a confidential manner simply using his public-key. Therefore, I would like to explore symmetric key cryptography to send messages to him, while retaining the advantage that I or anyone else can send a message to student A in a confidential manner simply using his public-key. How can I do this? HINT: establish a one-time symmetric key and share that with student each time I send a message. Elaborate on this, but importantly, use a block diagram to show what is the processing at sender & receiver ends.

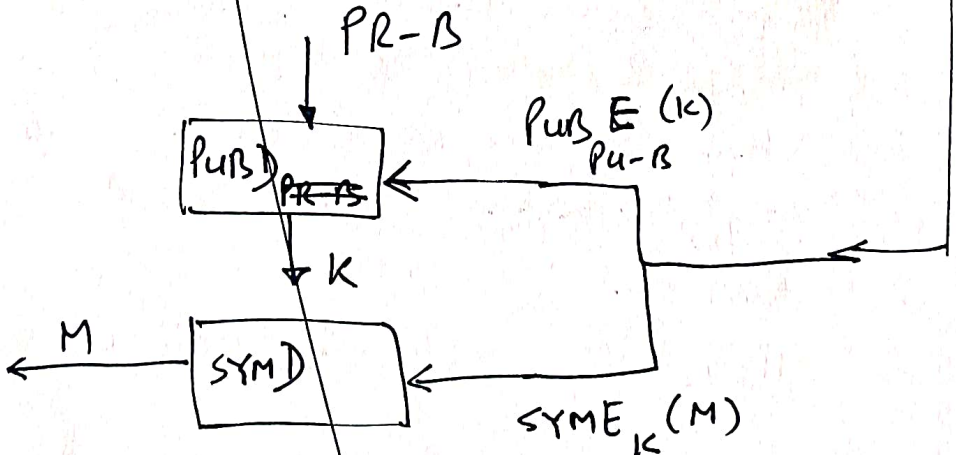
Marks on Q8

Sender A selects a symmetric key K to encrypt message M. Assume purpose of messaging is 'Confidentiality'. Sender A uses PU-B to encrypt the selected key, K, and sends the same together with the encrypted message, viz. SYM-E(M). At receiver's end User B uses his private key PR-B to decrypt & obtain K. It then use key K to decrypt the encrypted message, to obtain M.

Block diagram at sender's end:



Block diagram at receiver's end:



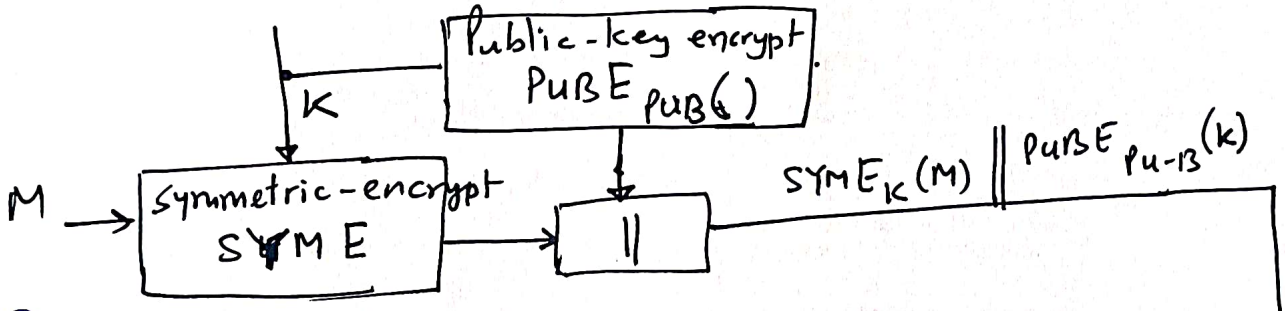
Marks on Q8: 12

8. Each PhD student has a RSA related <public-key, private-key> pairs, or <PU-A, PR-A>, <PU-B, PR-B>, etc. They have securely shared their public keys with me. I can now encrypt my messages and confidentially share with student A using his public key, PU-A. However, I find that encrypting a long message (such as 20 MB PPT file) using the public-key crypto-system takes a long time. The advantage of using public-key cryptography is anyone can send a message to student A in a confidential manner simply using his public-key. Therefore, I would like to explore symmetric key cryptography to send messages to him, while retaining the advantage that I or anyone else can send a message to student A in a confidential manner simply using his public-key. How can I do this? HINT: establish a one-time symmetric key and share that with student each time I send a message. Elaborate on this, but importantly, use a block diagram to show what is the processing at sender & receiver ends.

6

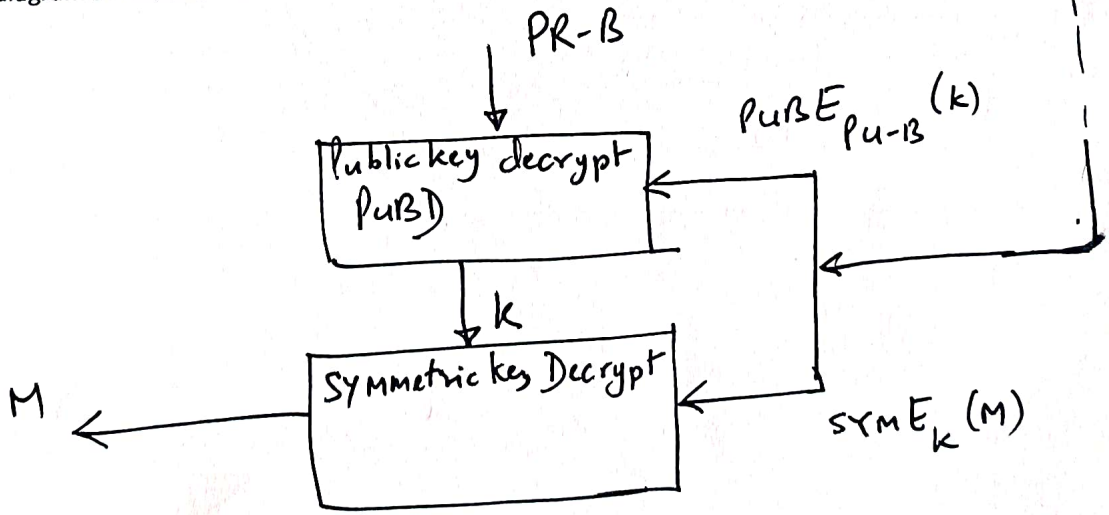
Assume purpose is 'confidentiality'. Sender A selects symmetric key K to encrypt message M. He uses public key of receiver B's public key, PU-B, to encrypt the selected key K, and sends the same together with encrypted message, viz $SYME_K(M)$. Receiver B uses his private key PR-B to decrypt and obtain K. He then uses key K to decrypt the encrypted message & obtains M.

Block diagram at sender's end:



3

Block diagram at receiver's end:



3

Marks on Q9: 12

9. Consider an RSA based public key cryptosystem, where n = 119. The **public** key used to encrypt a message is e = {5, 119}.

	Quotient, Q(i)	Remainder, R(i) R(i+1) = R(i-1) - Q(i+1)R(i)	S(i) S(i+1) = S(i-1) - Q(i+1)S(i)	T(i) T(i+1) = T(i-1) - Q(i+1)T(i)
0		Φ=~~r~~ 96	1	0
1		e =~~r~~ 5	0	1
2	19	~~r~~ 1	1	-19
3	5	0		
4				
5				
6, etc.				

a. If the message M = 15 is encrypted using the **public** key {5, 119}, then what is the cyphertext, C? (3)

$C = 15^5 \bmod 119 = 36$

b. Given that an intruder has access to the public key only, can he **in this case** determine the private key {d, n} used for decryption? If so, what is it? Give details below. You may like to use the table given above to compute {d, n}. (6)

Given $n = 119 \Rightarrow n = 119 = 7 * 17 \rightarrow p = 7, q = 17 \Rightarrow \Phi = 96$

Given $\Phi = 96$, the private key $d = 77$ since $d * e \bmod \Phi = 1$

or $77 * 5 \bmod 96 = 1$

c. Assume that the intruder is able to compute the decryption key {d, n} then what is the plaintext M corresponding to ciphertext C = 2. (3)

$M = C^d \bmod n = 2^{77} \bmod 119$

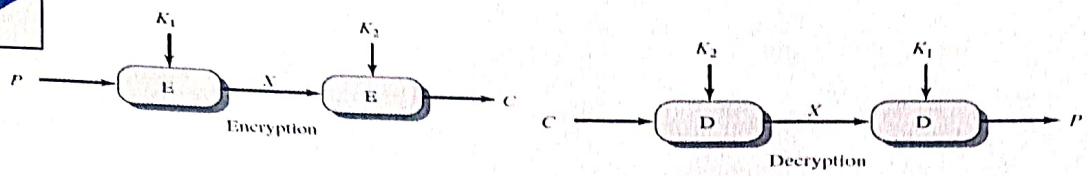
$= 2^{64} \bmod 119 * 2^8 \bmod 119 * 2^4 \bmod 119 * 2^1$

$= 86 * 18 * 16 * 2$

$= 32$

Marks on Q10: 12

10. Consider using 2 stages of DES. Unfortunately, an intruder has access to one pair of 64-bit plain text P, corresponding ciphertext $C = E_{K2}(E_{K1}(P))$. Here key K_1 is used in stage 1, and key K_2 is used in stage 2, as given below.



a. Describe how a brute-force attack with minimal effort may be launched on a 2-stage DES.

4

Given a plaintext, ciphertext pair (P, C)

1. for all possible keys K_1, compute $X_{K1} = E_{K1}(P)$
2. for all possible keys K_2, compute $Y_{K2} = D_{K2}(C)$
3. Look for a pair K_1 & K_2 such $X_{K1} = Y_{K2}$

b. What is the time complexity of the attack described in a. above, AND WHY?

4

Step 1: $O(2^{56})$; Step 2 = $O(2^{56})$; step3 : $2^{56} * \log(2^{56})$

Total complexity $O(58 * 2^{56}) = O(2^{56})$

c. Whether the attack described in a. above will absolutely positively determine the keys K_1 and K_2? Or many such attacks described in a. above would be necessary to increase one's confidence in the keys K_1 and key K_2 So discovered?

4

Step 1, 2, 3 will likely result in one pair.

~~There may be other~~ One will (or must) test the key pair (K1, K2) on other pair of plaintext, ciphertext (P2, C2), (P3, C3), etc. to gain confidence in the discovered key pair (K1, K2).